Software Engineering Lab

Lab 13

Backward Compatibility

Ubah Sistem,
Jaga Consumer Lama



lukman-ss

Satu nomor menjadi banyak nomor

Kebutuhan baru menambah daftar telepon per user. Consumer lama masih membaca phone sebagai string.

SEBELUM

users.phone dan field API phone menyimpan satu nomor. LegacyConsumerDTO mengharapkan string.

SESUDAH

user_phones dan field phones merepresentasikan banyak PhoneEntry, termasuk penanda primary.

MASALAH TRANSISI

Langsung mengganti tipe field lama membuat kontrak consumer berubah sebelum consumer siap.

Desain akhir yang baik tetap membutuhkan jalur transisi yang kompatibel.

Expand → Migrate → Contract

Parallel change menyediakan masa ketika bentuk lama dan baru tetap tersedia.

Tahap	Tujuan
Expand	Tambahkan representasi modern; pertahankan bentuk lama
Migrate	Pindahkan writes, data historis, dan pembaca
Contract	Pensiunkan dependency dan representasi lama

BATAS LAB

Runtime memakai MemoryStore. schema.sql adalah ilustrasi, bukan DDL yang dieksekusi terhadap database.

Pola memberi urutan kerja; keamanan tetap ditentukan implementasi.

Tambah phones, pertahankan phone

UserResponse mempertahankan scalar lama sekaligus menambahkan array modern.

```
type UserResponse struct {
ID int `json:"id"`
Name string `json:"name"`
Phone string `json:"phone,omitempty"`
Phones []PhoneEntry `json:"phones"`
}
```

KONTRAK

Jangan mengubah phone menjadi array pada payload yang masih dikonsumsi sebagai string.

Perubahan aditif tetap perlu diuji dengan consumer yang relevan.

Uji dari sisi dua consumer

Test serialisasi memberi satu enriched payload kepada DTO legacy dan modern.

LEGACY

json.Unmarshal berhasil; nilai Phone tetap +628123456.

MODERN

Array berisi dua nomor; entry pertama memiliki nomor utama dan IsPrimary=true.

BATAS BUKTI

Ini dua DTO Go pada fixture tertentu. Bukan bukti semua strict decoder atau SDK menerima field tambahan.

Decode sukses perlu disertai assertion nilai dan bentuk data.

Payload enriched ≠ dua endpoint identik

V1 dan V2 mengirim DTO terpisah. Test enriched payload memiliki cakupan yang berbeda.

Jalur	Response
GetUser / serialisasi test	UserResponse: phone + phones
GetUserV1	LegacyConsumerDTO: scalar phone
GetUserV2	ModernConsumerDTO: array phones

BENTUK KOSONG

phone kosong dapat hilang karena omitempty. Phones nil bisa menjadi null, sedangkan slice kosong menjadi [].

Kontrak mencakup field yang hilang, null, dan array kosong.

Write mode menentukan jejak data

Mode tulis dipilih saat CreateUser dipanggil. Jejak itulah yang menentukan data untuk rollback.

Write mode	Legacy	Modern
LegacyOnly	Scalar terisi	Tidak diisi
Dual	Scalar utama	Primary + extraPhones
NewOnly	Phone=nil	Daftar nomor

DETAIL

ExtraPhones tidak diproses di LegacyOnly. Kemampuan banyak nomor tidak ditambahkan ke consumer lama.

Mempertahankan kolom lama tidak cukup jika writer berhenti mengisinya.

Dual-write: batas atomicity harus jelas

CreateDual menyimpan user dan primary modern di bawah satu mutex MemoryStore.

YANG TERJAGA

Nomor utama tersedia di representasi lama dan baru pada create melalui CreateDual.

BATAS

Error extra write diabaikan. Seluruh operasi multi-phone tidak dibungkus satu transaksi database.

EXTRA PHONES

Service menambah nomor ekstra lewat SavePhoneEntry terpisah setelah CreateDual selesai.

Sebut unit operasi yang atomik; jangan memperluas klaim ke seluruh workflow.

Read mode memilih sumber scalar

GetUser tetap mengambil daftar modern sebelum menentukan nilai scalar phone.

Read mode	Sumber nilai phone
LegacyOnly	Pointer legacy jika tersedia
Fallback	Primary / entry pertama modern; jika kosong, legacy
NewOnly	Primary / entry pertama modern

IMPLIKASI

ReadLegacyOnly tidak otomatis menyembunyikan phones pada enriched response. GetModernUser mengambil array tersebut.

Nama mode tidak menggantikan pemeriksaan field yang benar-benar dikembalikan.

Data historis memerlukan backfill

Dual-write untuk create baru tidak otomatis memindahkan record yang sudah ada.

CURSOR

Worker mengambil ID > LastProcessedID dalam batch berukuran terbatas.

SKIP

Daftar modern yang sudah terisi dilewati. Backfill tidak menimpa mismatch atau memperbaiki seluruh drift.

KONDISI COPY

Legacy phone harus ada dan tidak kosong; daftar modern harus kosong.

Pisahkan migrasi data historis dari perubahan perilaku writer baru.

Idempotensi punya unit yang spesifik

SavePhoneEntry tidak menambahkan nomor yang sama dua kali untuk user yang sama.

```
for _, p := range s.userPhones[userID] {
if p.Number == number {
return p, nil
}
}
```

KUNCI LOGIS

Pencarian dilakukan pada entry milik user tersebut, berdasarkan Number.

BATAS MERGE

Entry yang sudah ada langsung dikembalikan. IsPrimary tidak diperbarui oleh retry ini.

Deduplikasi bukan kebijakan reconciliation untuk semua perbedaan data.

store.go • cuplikan SavePhoneEntry

Resumable masih di dalam memori

Checkpoint menyimpan LastProcessedID, TotalMigrated, dan IsComplete pada objek worker.

TEST

Sepuluh user diproses 3 + 3 lalu sisa 4. Run ulang worker selesai mengembalikan nol.

RESTART

Tidak ada checkpoint persisten. Test tidak mematikan proses lalu memulihkan cursor.

SESUDAH COMPLETE

Worker yang selesai tidak memindai lagi. Record legacy yang muncul belakangan tidak otomatis ikut termigrasi.

Worker selesai bukan bukti bahwa semua writer sudah meninggalkan legacy.

Fallback read juga melakukan write

Saat modern kosong, GetUser mengambil legacy untuk scalar dan menyimpan entry modern secara lazy.

MANFAAT

Scalar tetap tersedia bagi pembaca selama data historis belum dibackfill.

URUTAN

Slice phones dimuat sebelum lazy write. Implementasi tidak mengambil ulang slice setelah penyimpanan.

EFEK SAMPING

Read tersebut mengubah store. Error SavePhoneEntry pada jalur ini diabaikan.

Efek di storage dan isi response yang sama perlu diverifikasi terpisah.

Read pertama: scalar benar, array kosong

Lazy backfill mengisi store, tetapi response memakai slice yang dibaca sebelum write.

Titik pemeriksaan	Kondisi
Sebelum fallback	Modern store kosong
Response pertama	phone terisi; phones masih kosong
Store setelah read	Satu entry sudah ditambahkan
Read selanjutnya	Dapat membaca entry modern

Test fallback memeriksa scalar + store, bukan array response pertama.

Reconcile mendeteksi, tidak memperbaiki

Legacy phone dibandingkan dengan primary number modern sebelum contract.

DRIFT

Modern kosong atau primary berbeda dari scalar lama dihitung sebagai mismatch.

EKSEKUSI

Demo dan test memanggil fungsi secara eksplisit. Tidak ada repair atau scheduler audit otomatis.

CAKUPAN

User dengan Phone=nil dilewati. Setelah contract, fungsi langsung mengembalikan nol.

Nol drift pada pemeriksaan terbatas bukan bukti seluruh data modern valid.

Counter historis berbeda dari hasil scan

Return drifts dan counter DriftDetected memiliki arti waktu yang berbeda.

Nilai	Makna
Return drifts	Jumlah temuan pada scan ini
DriftDetected	Akumulasi temuan sepanjang umur counter
LegacyReadHits	Jumlah pemanggilan legacy, termasuk yang gagal

CONTOH INTERPRETASI

Mismatch yang sama dihitung lagi saat scan diulang. Current scan bersih dapat berdampingan dengan counter historis nonzero.

Baca unit, cakupan, dan horizon waktu sebelum memakai metrik sebagai gate.

Rollback dibatasi write history

TestRollbackScenarios membandingkan record dual-write dengan record new-only.

Asal record	Saat kembali ke legacy read
WriteDual	Nomor utama masih terlihat
WriteNewOnly	Nomor scalar kosong; read tidak error

ARTI KEGAGALAN

Nomor modern masih bisa ada. Yang gagal adalah visibilitas data melalui pembaca lama, bukan harus hilang dari semua storage.

Rollback aplikasi tidak otomatis mengembalikan representasi data lama.

Contract guard hanya melihat counter

Tanpa force, legacy hit yang pernah tercatat menghalangi contract. Tidak ada observation window bawaan.

```
if !force && s.obs.LegacyReadHits.Load() > 0 {
return fmt.Errorf("%w: recorded %d legacy reads",
ErrContractViolation,
s.obs.LegacyReadHits.Load())
}
```

BATAS

Tidak memeriksa backfill selesai, drift nol, atau writer lama sudah berhenti.

FORCE

Demo memakai ApplyContract(true). Bypass bukan bukti traffic benar-benar nol selama periode tertentu.

Counter nol dapat meloloskan contract meskipun data belum dimigrasi.

service.go • cuplikan ApplyContract; cmd/demo/main.go

Contract mengubah beberapa state

ApplyContract menjalankan langkah berurutan; seluruh transisi tidak menjadi satu operasi atomik service.

Urutan	Perubahan
1	Write mode → NewOnly
2	Read mode → NewOnly
3	ContractApplied → true
4	LegacyDropped; semua User.Phone → nil

Transisi concurrent memerlukan bukti lebih dari atomic field individual.

V1 pensiun, enriched field masih ada

Sesudah contract, GetLegacyUser error dan handler V1 merespons 410 Gone.

V2

Modern reads tetap memakai daftar user_phones yang sudah tersedia.

MAKNA CONTRACT

Implementasi menonaktifkan jalur V1 dan nilai storage lama, bukan menghapus seluruh field legacy dari source.

ENRICHED RESPONSE

Definisi UserResponse.Phone belum dihapus. ReadNewOnly dapat mengisinya dari nomor modern.

Periksa apa yang benar-benar dipensiunkan, bukan hanya nama fase.

SQL dan header masih ilustratif

Dua detail dokumentasi perlu dibatasi sebelum diangkat menjadi klaim production.

SCHEMA

SQL expand mempertahankan users.phone NOT NULL. MemoryStore CreateModern memakai Phone=nil; urutan SQL new-only belum lengkap.

HEADER

Deprecation dan Sunset hardcoded. Test hanya memeriksa true dan nonempty, bukan kepatuhan standar atau scheduler tanggal.

BATAS OPERASIONAL

Tidak ada eksekusi DDL, pengukuran lock database, atau bukti zero downtime di MemoryStore.

Contoh mekanisme tidak otomatis membuktikan kesiapan operasi production.

Test yang ada dan yang belum terbukti

Unit dan integration test menguji jalur penting, dengan batas assertion yang tetap perlu dibaca.

CAKUPAN

Decode dua DTO, batch backfill, fallback, drift, handler 200/410, lifecycle, dan dua skenario rollback.

CONCURRENCY

Writer, reader, backfill, reconciliation berjalan overlap; banyak error diabaikan dan assertion akhir hanya DualWriteErrors.

BELUM DIBUKTIKAN

Semua record konsisten, durable resume, dan concurrent contract yang aman. Test dibaca, belum dijalankan di sini.

Race detection pada jalur yang berjalan bukan pengganti assertion semantik.

Jaga transisi sampai dependency selesai

Gunakan modul lab untuk reproduksi. go.mod menyatakan Go 1.26.7; Go tidak tersedia di lingkungan ini.

```
cd labs/13-backward-compatibility
go test -v ./...
go test -race ./...
go run ./cmd/demo
```

MENTAL MODEL

Compatibility must survive the transition, not just the final state.

URUTAN KEPUTUSAN

Pertahankan kontrak lama → migrasikan data dan consumer → kumpulkan bukti → contract setelah siap.

Perintah ini instruksi reproduksi, bukan laporan hasil eksekusi baru.

Ubah sistem. Jaga transisinya.

Compatibility must survive the transition, not just the final state.

Software Engineering Lab 13 - Backward Compatibility
internal/compat/*.go • tests/*.go
cmd/demo/main.go • schema.sql • README.md • go.mod

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/13-backward-compatibility`

Lukman / lukman-ss

Sumber: commit 82af988b89439fa92b1200406c5afbd59ce72a56
Test dibaca, tidak dijalankan.
MemoryStore: bukan bukti migrasi database production.